Data declaration:.

1byte = 8bits

To declare a byte in ram:.

db 65 | db 'A'

↓

define byte

(Allocates 8 bits in memory)

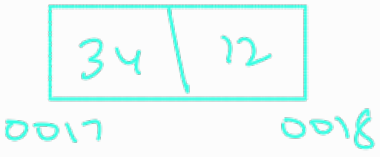
dw 0x1234 16bits

↓

define word

Allocates 16 bits in memory.

Because it is allocation in Ram it follows little endian



Ram

1 digit = 4nibbles.

2digit = 1byte

OR

db 0x34, 0x12 → nibble

↓ ↓

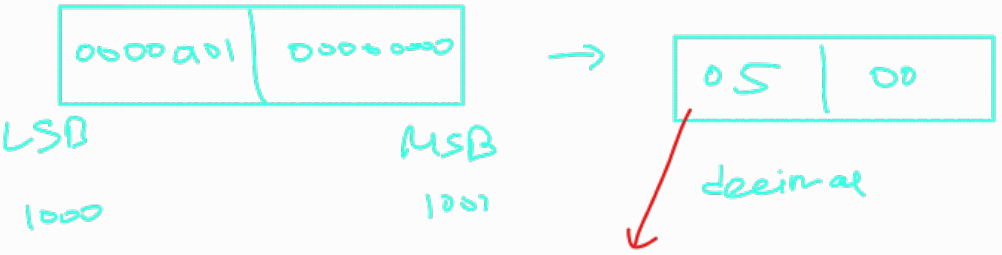
Stores 32 bits. nibble

In Assembly we declare data at end

dw 5



Dividing into 2 groups as smallest we can store is 1byte.



Let say address

adding zeros to make it 1byte

```
num1 : dw 5      0500
```

Label

(give name to the memory location)

→ Reserve 16 bits in ram and store 5 in it in little endian format.

```
mov ax, [num1]
mov bx, [num2]
add ax, bx
mov bx, [num3]
add ax, bx
mov [num4], ax
```

↓

result of three no is stored in num4

Declaration		value stored at ram
num1 : dw	5	0500
num2 : dw	10	0A00
num3 : dw	15	0F00
num4 : dw	0	0000

[num1] → means go to num1 memory location and get the value stored at it.

num2 → get me the memory location value only.

mov num4, ax (X)

we cannot put a value inside a memory location. Because memory is a constant.

mov 5 ax ✗ illegal / cannot put a constant as a destination.

```
[org 0x0100]
mov ax, [num1]
mov bx, [num1+2]
add ax, bx
mov bx, [num1+4]
add ax, bx
mov [num1+6], ax
mov ax, 0x400
int 21h

num1 dw 5, 10, 15, 0
```

Note: A word in Ram is of two bytes.

05	00

We use +2 bytes to move to next 16 bit number in memory.

if we did db 5 → (0000 0101) now to access the next no we do +1, ... +2, +3.

using half registers to perform calculation on 8bits.

```
mov al, [num1]
mov bl, [num1+1]
add al, bl
mov bl, [num1+2]
add al, bl
mov [num1+3], al

num1 db 5, 10, 15, 0
```

we use half register as numbers were in 8bits

Valid Instruction

Reg → Reg

```
mov ax, bx
add ax, bx
mov ax, bL   (X)
```

Reg → memory

```
mov [num1], ax   ✓
```

↓

Registers define size

Reg → constant

```
mov 5, ax   (X)
```

memory → Reg

```
mov ax, [num1]   ✓
```

Registers will define size

memory → memory

```
mov [num1], [num2]   X
```

Data Bus is uni-directional

constant → memory

mov [num1], 5 ✗

mov byte [num1], 10 ✓

mov word [num1], 10 ✓

makes the diff b/w

mov byte [num1], 10 VS

db 10 is fast

the latter we can always change

to value (10 → Another value)

this is happening during runtime

But for dB ID I

have to overwrite the memory.

db 5 (8 bits) ✓

db 10 (8 bits) ✓

db 121 ✓

128 64 32 16 8 4 2 1

0 1 1 1 1 0 0 1

↓

decimal by default

db 300

256 128 64 32 16 8 4 2 1

(1 0 0 0 1 1 1 0 0) X

9 bits cannot store

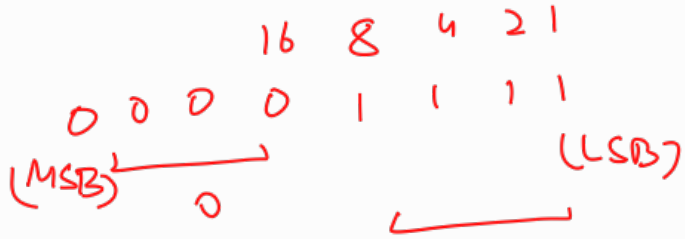
db 132h (X) cannot store

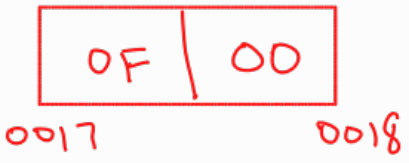
4+4+4 → 12 bits

Note:: Define data always at the end of your program → code comes first.

Size Mismatch errors.

DW 15





Stored in ram

DB 15





in 1byk DB there is no little endian Swapping.

Little endian only swaps multi byk values (>byk, 4byles)

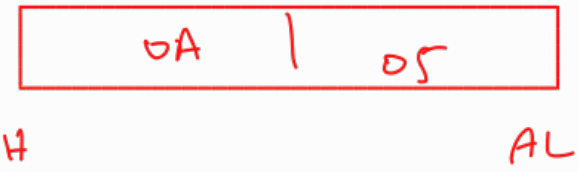
mov ax, [num1] Ax -> 16bit reg

num1 : DB 5 05h

DB 10 0Ah

In AX 0A05h is stored because Ax is 16bit but it grabs two bytes. Even though we wanted it to store 5 only.

mov ax, [address] → fetch two bytes from memory



05 (LSB)
0A (MSB)

Because 0A is at a higher address than 05 (comes below 05)
So it is MSB. and 05 is LSB

mov al, [num1] AL reads 1 byte

mov ax, [num1] Ax reads 2 bytes

mov byte [num1], 5 Stores 1 byte

move word [num1], 5 Stores 2 byte

(Indirect addressing)

```
mov al, [num1]
mov bl, [num1+1]
mov cl, [num1+2]
```

Direct addressing using labels.

now Registers will hold the addresses

Bx , BP , SI , DI → these Registers will hold the addresses.

e.g

Bx stores the base address of num1.

[Bx] we can read num1 [Bx+1] we can read num2 and SI on.

```
dw 5,10,20
mov bx, num1
mov ax, [bx]
add bx, 2
add ax, [bx]
```

Bx now stores address of num2

go to memory address in Bx

mov bx to second number. Each number is 2 bytes apart

if it was db 5 then we would do `add bx, 1` to get the next num.

Program to add 10 numbers.

bx → holds address

[bx] → gives us value

dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50

bx points to address

```
mov bx, num1
mov cx, 10
mov ax, 0
```

→ counter loop runs 10 times

→ initialise sum to 0

```
Li:
↓
Label
```

```
add ax, [bx]
add bx, 2
Sub cx, 1
Jnz 12
```

add num to ax

move to next no.

Decrement

Jump if not zero.

(if the last operation. Sub cx,1 answer is not zero keep doing the Loop) else exit.

```
mov [total], ax
```

↓

writing total in mem

Jump back to 11 if cx ≠ 0

Uses 2 F.

Direct Addressing [num1] → hardcoded value

Indirect Addressing [BX] → using a Reg

Segment Association (which Seg is associated with which offset)

to calculate the phy. add

CS :	IP
DS :	SI, DI, BX
SS :	BP, SP

Seg → offset.

0010 + 0030 →

0042 + (0030)

CS : 0100h DS : 0020h SS : 0013h

BX : 0200h BP : 0010h DI : 0030h

[BP + DI + 0x0002]

default Seg we will use

SS → Seg of BP

---

e.g DS:- 2000h

BX:- 0100h

mov ax, [BX] read value at DS:BX

(2000 x 16) + 0100 = 20100h phy. Address

note: mov ax, [bx] cpu would use by default DS Register.

mov ax, CS:[BX] override DS with CS

DS = 2000h

BX = 0050h

mov ax, [CS:BX]

CS = 1000h

physical = (1000 x 16) + 0050

= 10050h

e.g

CS:- 1000h

BX = 0100h

SI = 0200h

offset = 0700h

mov ax, [CS: BX + SI + 0700]

offset = 0700 + 0100 + 0200
= 0A00

$$phy = (1000 \times 10) + 0A00 = 10A00h$$

Wrap around case:

[BP + SI]

BP ①9000
SI 7100
0100

SS 0200
0100
0210 0

if ans exceeds the bit drop the carry

Note

[BX + BP + 0X0700] → we cannot use two Base pointers at the same time (use only one)

Invalid

[SI + DI + 0X7001]

↳ we cannot use Index pointers at the same time

Part of reg cannot be used for memory addresses

[AL + AH]

(X)

general purpose reg cannot be used for address cal

[Bx + Ax] or [Bx + Dx]

Ax, Cx, Dx → GPR [Ax] [Cx] [Dx] (X)

[BP-2] valid [BP-52] invalid

we can only Subtract constants

only allowed Reg to put []

BP, SI, DI, Bx or Some label

CS : IP
DS : SI, DI, Bx
SS : BP, SP

Seg offset

Base + Index + Offset

general form

[IP] and [SP]

cannot be used